

Chapter X: Architecture and Process Model of the RS-DRL-Based Self-Adaptive System

Thesis Documentation

December 27, 2025

1 Chapter Overview

Self-adaptive systems operating under uncertainty require not only effective runtime decision making but also clear architectural separation between monitoring, learning, decision, verification, and execution concerns. While the reference RS-DRL architecture introduces these elements at a high level, its original representation does not explicitly capture the internal process logic and control flow governing their interaction.

This chapter presents a **systematic process-oriented refinement** of the RS-DRL-based approach. The goal is to provide a clear and analyzable description of how runtime data is transformed into adaptation decisions and how learning, verification, and control are orchestrated within the MAPE-K framework.

Figure 1 introduces a diagram map that organizes the process models presented in this chapter. The central integration diagram provides a high-level view of system behavior, while five detailed activity diagrams refine individual subprocesses. Together, these figures offer a complete, traceable, and modular representation of the proposed approach.

2 Diagram Map and Abstraction Levels

Figure 1 serves as a navigation aid for the process models presented in this chapter. The central integration diagram represents the overall runtime behavior of the system, capturing the closed-loop interaction between monitoring, analysis, planning, verification, learning, and execution.

Each surrounding diagram refines a specific concern:

- **D1** details the RS-DRL training and reward shaping process.
- **D2** describes runtime adaptation option prediction.
- **D3** refines the MAPE-K adaptation control logic.
- **D4** presents the adaptation option verification process using runtime models.
- **D5** details monitoring and state construction.

This hierarchical organization ensures that complexity is managed through progressive refinement rather than monolithic representation.

3 Integrated Process View

Figure 2 presents the **integrated process view** of the RS-DRL-based self-adaptive system. The figure refines the architectural view introduced in the reference model by making control flow, data flow, and feedback loops explicit.

The process begins with the managed system executing its normal behavior and generating runtime data. This data is continuously collected and transformed by the monitoring and state construction process before being forwarded to both the Analyzer and the RS-DRL components. The Analyzer evaluates whether adaptation goals are violated or at risk. When no violation is detected, the system continues normal operation without triggering adaptation.

Diagram Map of RS-DRL Process Models

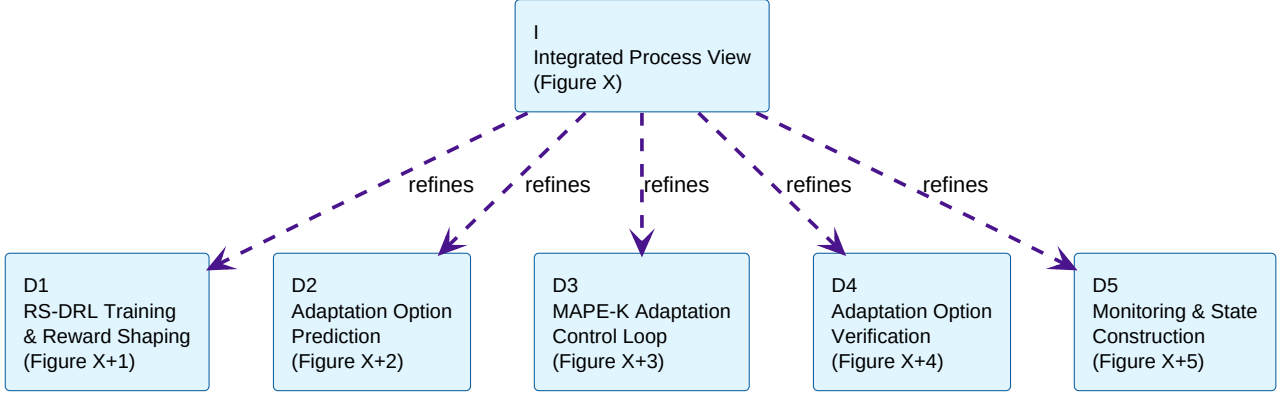


Figure 1: Diagram map of the RS-DRL-based self-adaptive system. The central integration diagram provides a high-level process view, while Diagrams D1–D5 refine the learning, decision-making, verification, control, and monitoring subprocesses presented in this chapter.

If adaptation is required, the Planner invokes the adaptation option prediction process, which relies on a previously trained RS-DRL model. Candidate adaptation options are evaluated through runtime model verification, and the resulting quality estimates are used both to justify adaptation decisions and to generate learning feedback. The selected adaptation is then executed on the managed system, closing the control loop.

4 Monitoring and State Construction

Figure 3 details the monitoring and state construction process. This process represents the **entry point of uncertainty** into the system and plays a critical role in enabling effective learning and decision making.

Raw runtime data is collected from both the managed system and its environment. This data is pre-processed through filtering, normalization, and missing-value handling to reduce noise and stabilize input distributions. Feature extraction then derives goal-relevant attributes while reducing dimensionality.

The resulting features are assembled into a state vector that serves as a contract between monitoring, learning, and planning. The constructed state is forwarded to the Analyzer for goal evaluation, to the RS-DRL module for learning or prediction, and to the Knowledge Repository for historical reference.

5 RS-DRL Training and Reward Shaping

Figure 4 presents a detailed view of the RS-DRL training process, which constitutes the core contribution of the proposed approach.

Training proceeds in episodes. At each step, the current state is used to select an action according to an ε -greedy policy. The action is executed in the environment, producing a reward and a subsequent state. Experiences are stored in a replay buffer, and learning is triggered only when sufficient experience has been accumulated.

A distinguishing feature of this process is the **novel reward shaping mechanism**, applied during minibatch sampling. Failed experiences are selectively reinterpreted as optimistic successes according to a configurable ratio. Importantly, reward reshaping is applied only to the sampled minibatch and does not modify the replay buffer itself. This design mitigates local optima while preserving experience diversity.

Figure 5 isolates the proposed reward shaping mechanism applied during RS-DRL training. Unlike conventional reward shaping techniques that globally modify the reward function or alter stored experiences, the proposed approach operates exclusively at the minibatch level during learning.

After sampling a minibatch from the replay buffer, each experience is evaluated to determine whether it represents a failure transition. Failed experiences are then probabilistically reinterpreted as optimistic successes according to a reshaping probability p . Importantly, reward reshaping is applied only to the sampled minibatch and does not alter the contents of the replay buffer. This design preserves the diversity

and integrity of accumulated experiences while mitigating the dominance of failure transitions during early learning stages, thereby accelerating convergence without biasing long-term policy optimization.

The reshaped experiences are used to update the Q-network via gradient descent, after which the trained model is stored in the Knowledge Repository for later use by the planner.

6 Adaptation Option Prediction

Figure 6 illustrates the runtime adaptation option prediction process. Unlike the training process, this path is strictly **non-learning** and is invoked only when adaptation is required.

Upon notification from the Analyzer, the Planner retrieves the trained RS-DRL model and evaluates the current state against all available adaptation options. The option with the highest expected value is selected, typically using a purely exploitative strategy or minimal exploration.

The predicted adaptation option is then forwarded to the Executor for enactment. This separation ensures that computationally expensive learning activities do not interfere with runtime responsiveness.

7 MAPE-K Adaptation Control Loop

Figure 7 refines the adaptation control logic using the MAPE-K framework. The figure emphasizes that adaptation is **event-driven** rather than continuous.

The Monitor observes system behavior and updates the Knowledge Repository. The Analyzer evaluates the observed state against adaptation goals and determines whether intervention is required. If no violation is detected, the system continues normal operation.

When adaptation is triggered, the Planner delegates decision making to the RS-DRL-based predictor rather than performing direct search or optimization. The Executor then applies the selected adaptation to the managed system, after which monitoring resumes. This design preserves a clear separation between control logic and learned intelligence.

8 Adaptation Option Verification

Figure 8 presents the adaptation option verification process, which provides semantic grounding and trustworthiness for learning-based decisions.

Candidate adaptation options are evaluated using runtime models parameterized with the current system state. Statistical model checking, implemented via UPPAAL-SMC, is used to estimate the probability of satisfying adaptation goals under uncertainty. Extracted quality metrics are aggregated into either a scalar reward (during training) or a quality estimate (during planning).

By mediating between architectural models and learning, verification ensures that rewards and decisions remain aligned with system-level objectives.

9 Summary and Traceability

This chapter presented a structured process model of the RS-DRL-based self-adaptive system. Starting from a high-level integration view, the system was progressively decomposed into monitoring, learning, prediction, verification, and control subprocesses.

Each diagram refines a specific concern while maintaining traceability to the reference architecture. Together, these models provide a clear foundation for implementation, analysis, and evaluation of the proposed approach under uncertainty.

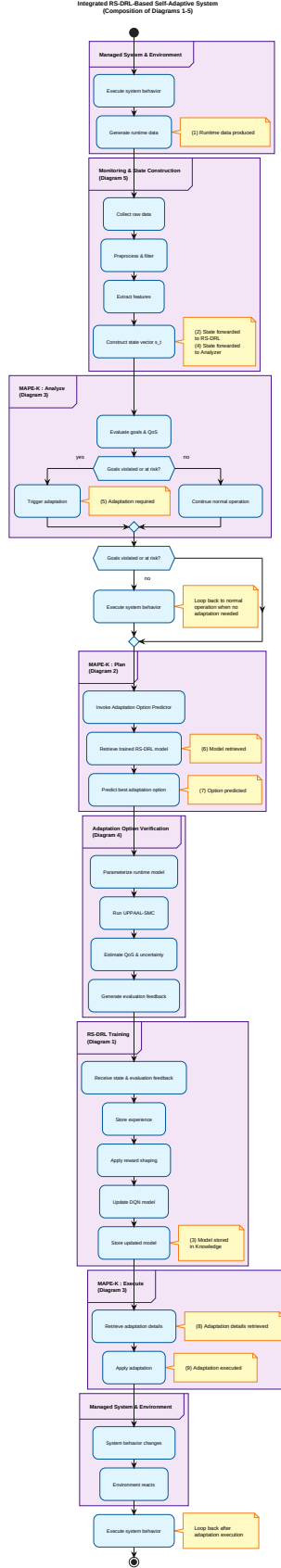


Figure 2: Integrated process view of the RS-DRL-based self-adaptive system. The figure illustrates the closed-loop interaction between monitoring, learning, verification, planning, and execution, and refines the original architectural view presented in Figure 4 of the reference model.

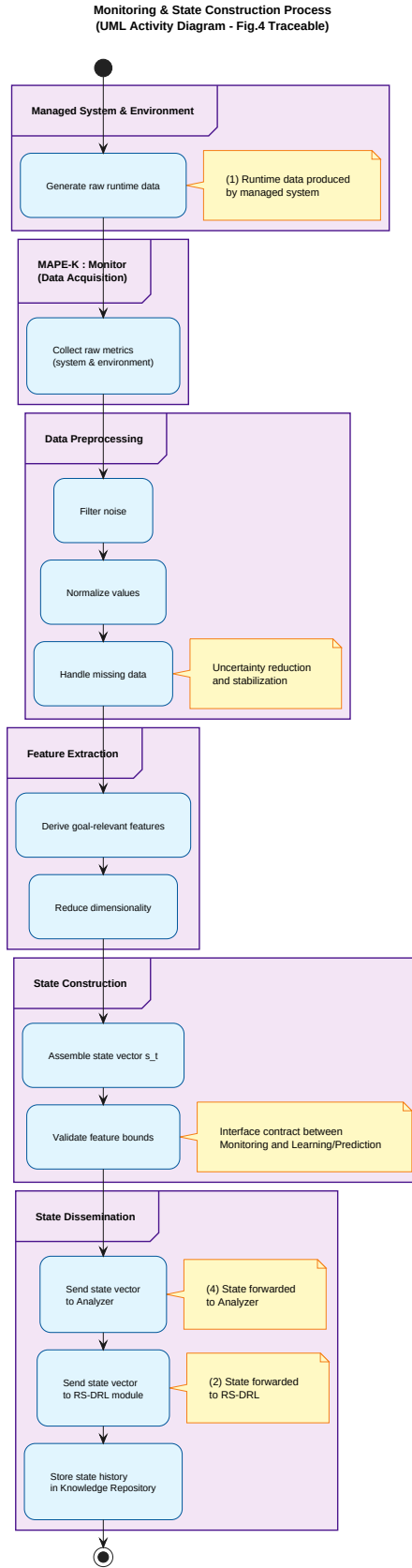


Figure 3: Monitoring and state construction process. Raw runtime data from the managed system and environment is filtered, abstracted, and transformed into a state vector suitable for both RS-DRL learning and runtime adaptation analysis.

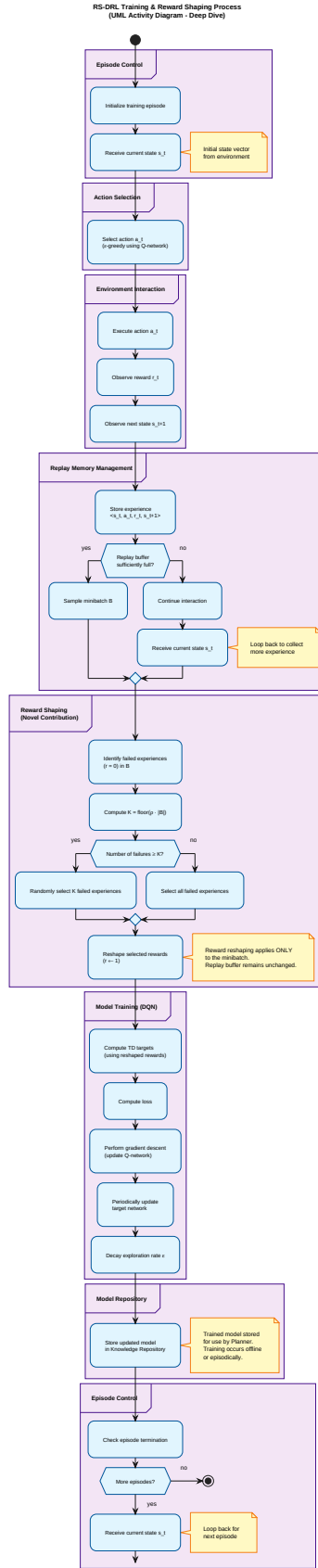


Figure 4: RS-DRL training and reward shaping process. The figure details the episodic interaction with the environment, experience replay, and the novel reward reshaping mechanism applied during minibatch learning.

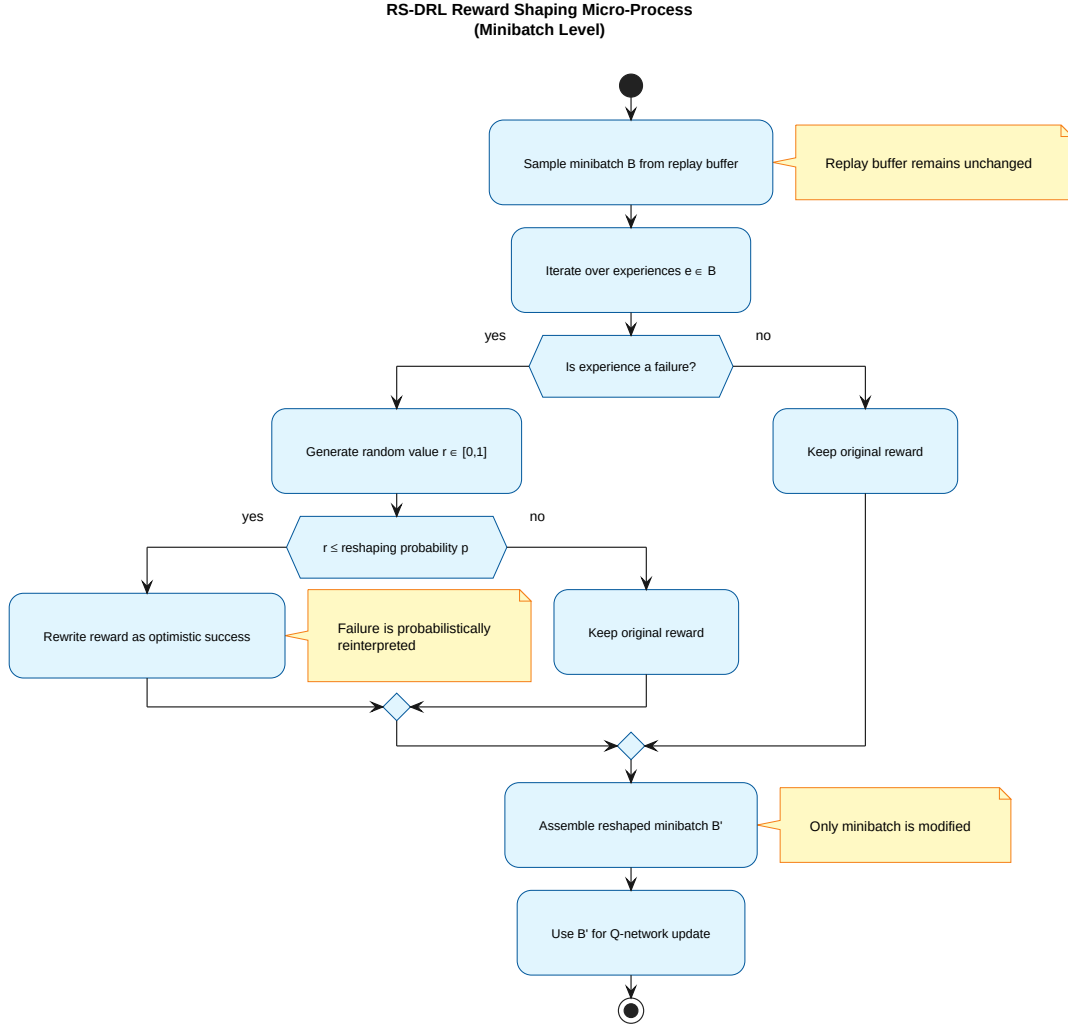


Figure 5: Reward shaping micro-process applied during RS-DRL training. Failed experiences sampled in a minibatch are probabilistically reinterpreted as optimistic successes according to a reshaping probability p . Reward reshaping is applied exclusively to the sampled minibatch and does not modify the replay buffer, preserving experience diversity while accelerating learning convergence.

**Adaptation Option Prediction Process
(Runtime Decision Path - Fig.4 Traceable)**

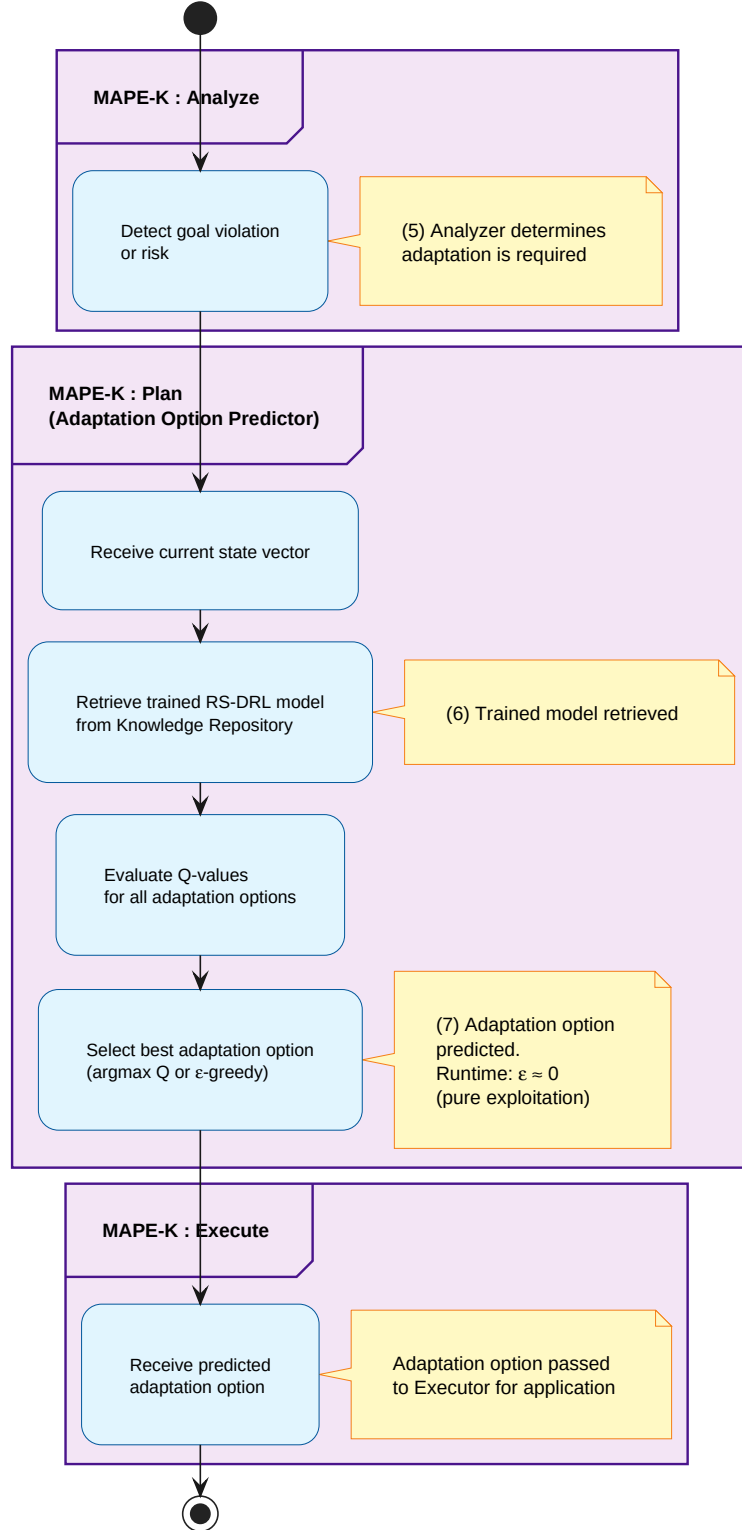


Figure 6: Adaptation option prediction process at runtime. Upon detection of goal violations, the trained RS-DRL model is used to predict the most suitable adaptation option without further learning.

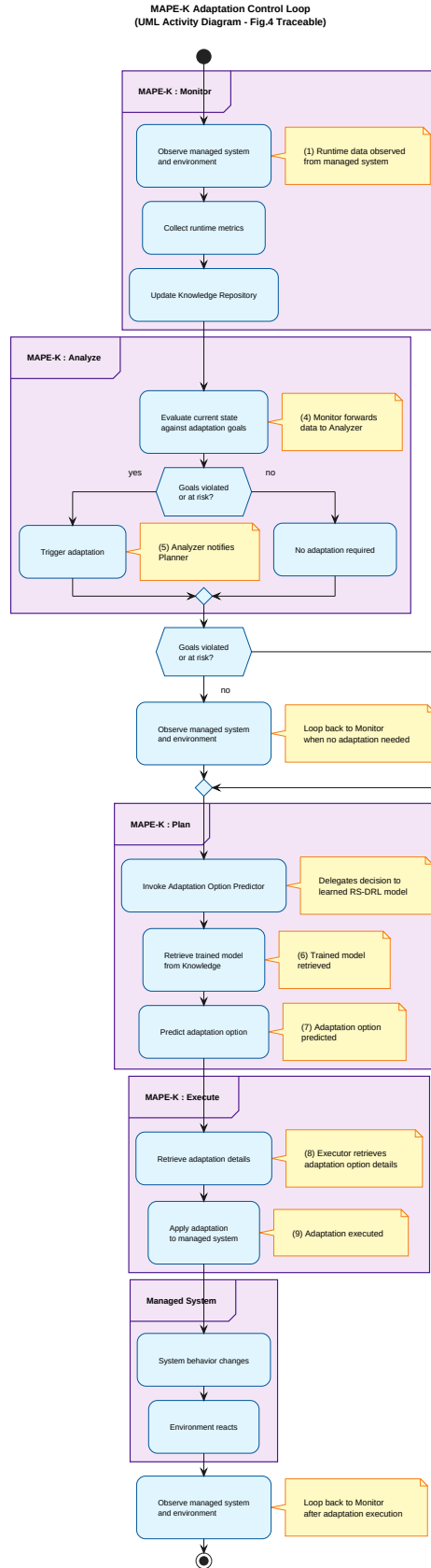


Figure 7: MAPE-K adaptation control loop. The figure illustrates how monitoring and analysis trigger adaptation, how planning delegates decision making to the RS-DRL predictor, and how execution enacts the selected adaptation.

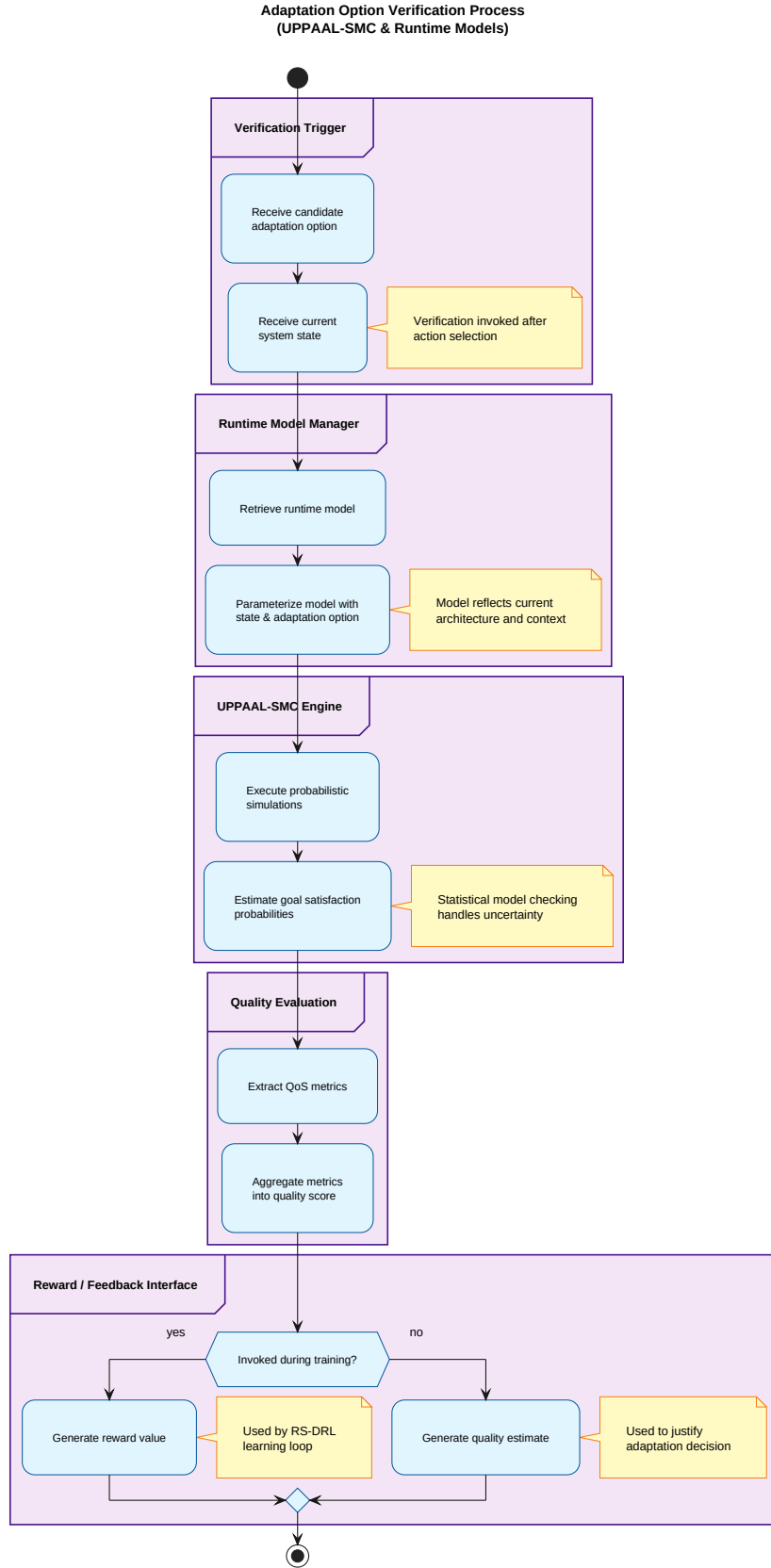


Figure 8: Adaptation option verification using runtime models and statistical model checking. The verification process evaluates the expected quality of adaptation options under uncertainty and provides feedback for learning and decision justification.